POWER BI ARCHITECTURE NOTES

DAX Calculation & Data Modeling Guide

Comprehensive technical breakdown of multi-granularity aggregation, DAX evaluation contexts, and Dimensional Modeling principles.

1. The Problem Core: Why `RELATED()` Fails

In Power BI, the `RELATED()` function is designed to fetch values from a related table following a **1-to-Many** or **1-to-1** relationship along the direction of the relationship line (from the "One" side to the "Many" side).

Technical Root Cause

The `RELATED()` function requires a unique primary key on the target table. Because `Product ID` is duplicated across multiple rows in **WarehouseCosts** (e.g., `CD-7907090` has 6 separate material rows for Ghee, Pepper, Rice, Salt, Chicken, Cheese), Power BI cannot establish a standard 1-to-Many lookup relationship. If attempted, DAX throws a runtime error or produces invalid evaluation contexts because it cannot determine *which single row* to return.

2. Key DAX Function Definitions & Core Concepts

ADDCOLUMNS

Definition: An iterator function that returns a table with new columns added to the specified input table.

Concept: It iterates row-by-row over the source table (`DetailedData`). Inside `ADDCOLUMNS`, a **Row Context** is active for each row being evaluated.

CALCULATE

Definition: Evaluates an expression in a modified filter context.

Concept: It is the only DAX function capable of performing **Context Transition** (converting an existing Row Context into an equivalent Filter Context) and applying filter overrides.

VAR / RETURN

Definition: DAX block used to store calculation expressions as named variables.

Concept: Variables are evaluated *once* at the moment of declaration within their scope. They simplify complex calculations, improve readability, and boost execution performance.

SUM

Definition: Aggregation function that adds all numbers in a specified column.

Concept: Operates over a column in a **Filter Context**. When combined inside `CALCULATE`, it sums only the rows matching the filter predicate.

3. Data Model Architecture: Fact vs. Dimension Tables

Understanding how tables function within a modern business intelligence schema is essential to writing performant DAX code.

Architectural Pillar	Fact Tables (e.g., `DetailedData`)	Dimension Tables (e.g., `DimProduct`)
Primary Role	Records business events, transactions, and numeric measurements (Volume, Revenue, Quantity).	Provides descriptive attributes and context used for slicing, dicing, and filtering facts.
Key Characteristics	High row count, foreign keys linking to dimensions, frequently updated, contains additive measures.	Unique surrogate/business keys (1 row per entity), descriptive text attributes, lower row count.
In Your Scenario	`DetailedData` contains physical shipment transaction events with Volume data.	`WarehouseCosts` acts as a detail fact table (or multi-row bridge) containing line-item warehouse breakdown costs per product.
Best Practice	Avoid direct Many-to-Many calculations where possible; aggregate through intermediate Dim tables or bridge tables.	Filter facts by filtering dimensions; model relationships as 1-to-Many from Dim to Fact.

4. DAX Solution & Code Execution Breakdown

Below is the recommended DAX calculated table formula using physical shipment volume (**Units**) multiplied by total aggregate warehouse cost per product.

```
ProductVolumeWarehouse =
ADDCOLUMNS(
    'DetailedData',
    "Warehouse Cost",
        VAR ProductID = 'DetailedData'[Product ID]
        RETURN
            CALCULATE(
                SUM('WarehouseCosts'[Cost at warehouse in USD]),
                'WarehouseCosts'[Product Id] = ProductID
            ),
    "Volume × Warehouse Cost",
        VAR ProductID = 'DetailedData'[Product ID]
        VAR Volume = 'DetailedData'[Volume of material to be shippped in units]
        VAR WarehouseCost =
            CALCULATE(
                SUM('WarehouseCosts'[Cost at warehouse in USD]),
                'WarehouseCosts'[Product Id] = ProductID
            )
        RETURN
            Volume * WarehouseCost
)
```

Step-by-Step DAX Execution Walkthrough

- **Row Context Initiation:** `ADDCOLUMNS` iterates through each individual transaction row in `DetailedData`.
- **Variable Assignment:** `VAR ProductID` captures the scalar `Product ID` value for that specific row (e.g., `CD-7907090`).
- **Filter Modification:** `CALCULATE` filters `WarehouseCosts` where `WarehouseCosts[Product Id]` equals `CD-7907090`.
- **Aggregation:** `SUM` adds all material costs across the filtered rows (39 + 10 + 54 + 24 + 24 + 48 = $199).
- **Scalar Multiplication:** The computed total warehouse cost ($199) is multiplied by the row's physical unit volume (e.g., 17,200 units), yielding the final scalar output (3,422,800).